



Big Data

- Big Data
- 3Vs
- More Vs
- Distributed File Systems
- Sharding
- Master-Slave
- Sharding & Master-Slave
- Peer-to-Peer
- ACID vs. CAP
- NoSQL
- Graph DBs
- Aggregate DBs
- Key-Value DBs
- Document DBs
- Column family DBs
- SQL vs. NoSQL
- MapReduce

Credits



Some of the material contained in these slides is taken or adapted from the following sources

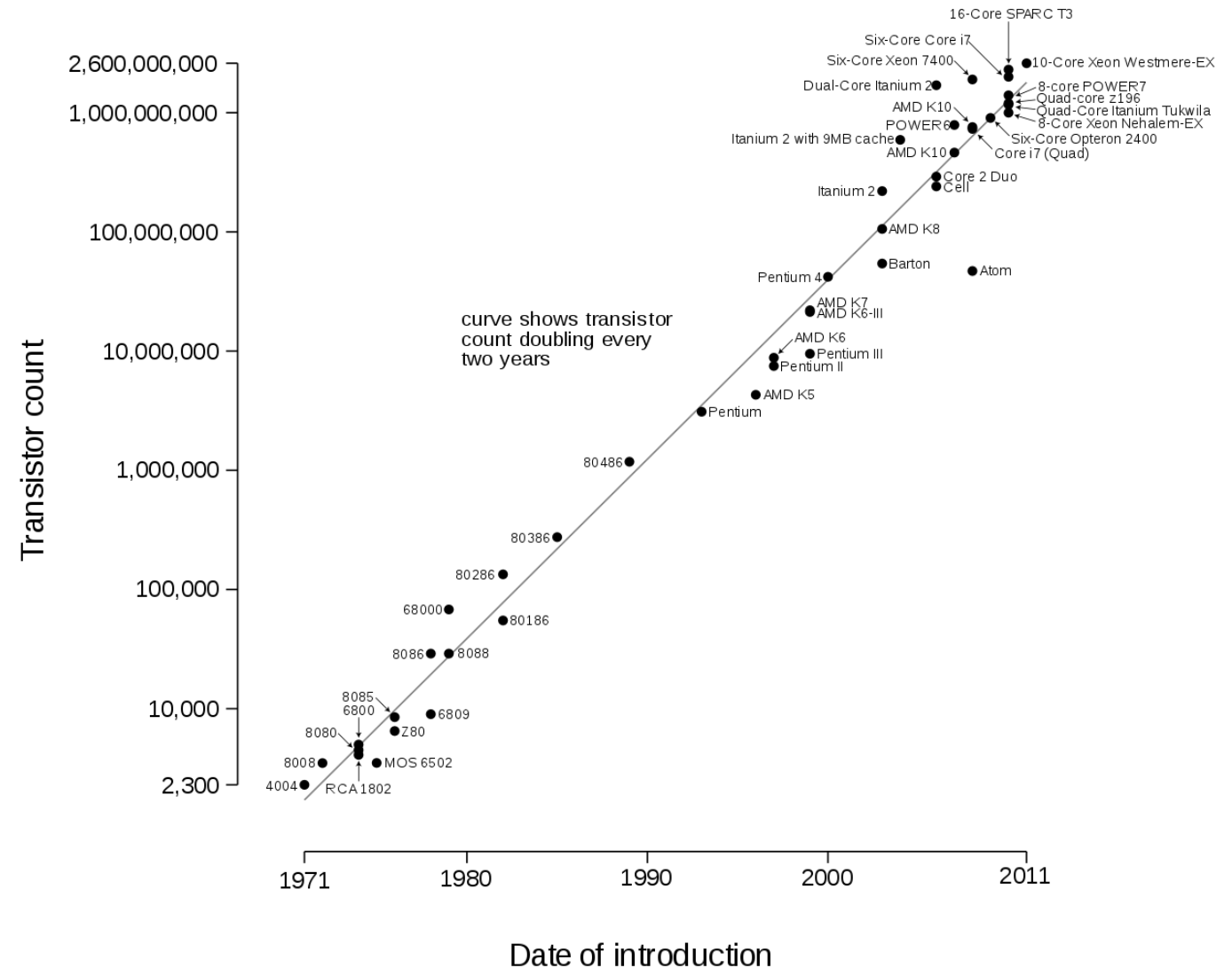
- D. Lembo, R. Rosati, “Data Management for Data Science”, Sapienza Università di Roma
- J. Leskovec et al., “Mining Massive Datasets”, Stanford University, <http://www.mmds.org>

Big Data

Nowadays, **digital information** is being created at stored at a rate that was **unthinkable** years ago

- More computational power
- Larger (and cheaper) storage resources
- Ubiquitous computing

Microprocessor Transistor Counts 1971-2011 & Moore's Law



Big Data

Moore's Law of Big Data:

"The Amount of Nonsense Packed
Into the Term "BIG DATA" Doubles
Approximately Every Two Years."

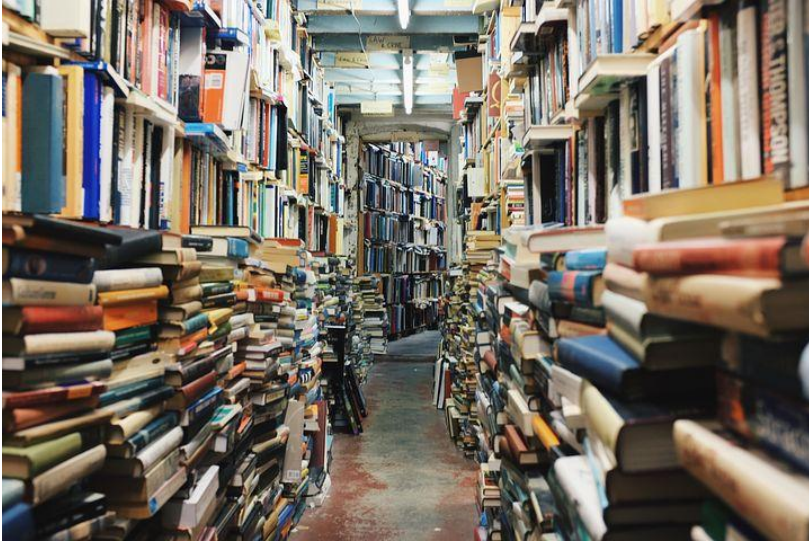
-Mike Pluta, 2014-08-10

Big Data is...

- Data that exceeds the processing capacity of conventional database systems (Dumbill, 2013)
- The capability to extract valuable information from large volumes of data

This *does not necessarily implies massive datasets*, but also **heterogeneous** and **non-tabular** ones

The Three Vs



Volume



Velocity



Variety

Volume

Huge amounts of data:

- Terabytes or even Petabytes
- Growing incredibly fast

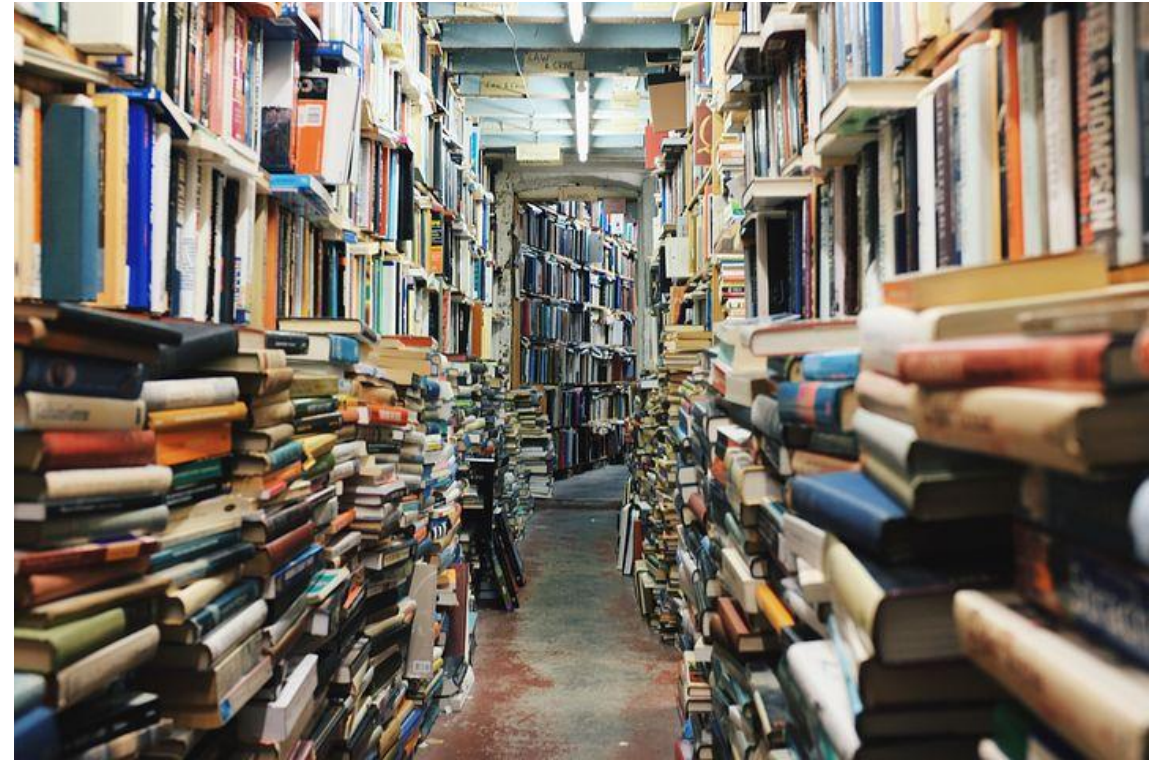
Amount of data **created by year**... :

- ~800 TB in 2000 (8×10^{14})
- ~500 EB in 2009 (5×10^{20})
- ~35 ZB by 2020 (3.5×10^{22})

Google + Amazon + Microsoft + Facebook ~
1200 PB

Beyond traditional storage:

- Large number of information **sources**
- Centralized approaches **cannot be enough**
- Towards **distributed systems** and **clusters**



Velocity

Rate at which data is collected

- Patterns similar to volume
- **Pervasive** and **ubiquitous** computing
- **Never-ending** flow of information
- Process information **as soon as it is available**
- **Stream processing**

Data may be **too fast to store** it all

- Pre-processing **on-the-fly**
- Store only a **portion** of the data that is produced
- Store only **some aggregation** of data



Variety

Heterogeneity of data

- Type of information (text, images, videos, ...)
- Type of representation (intensional¹ / extensional²)
- Data format (structured / unstructured)

¹ i.e., representing data using the rules that classifies them, e.g., all the people with an age higher or equal to 30. Used in contexts where the set of data is too big or in continuously changing.

² i.e., defining data listing explicitly all the members of a defined group. E.g., representing people in a database explicitly providing their names, IDs or records.



Veracity

A fourth V...?

- How to deal with the **quality** of the data?
- Can all this data be **trusted**?
- How to remove **noise**?
- Poor data quality **costs**!



Up to 10 Vs

More Vs...

- **V**ariability
- **V**alidity
- **V**ulnerability
- **V**isualization
- **V**alue
- ...

What can we do when dealing with data collections that:

- do not have a **pre-determined and fixed structure**?
- are **heterogeneous**?
- exceed the **storage** and **analysis** capabilities of *traditional relational database systems*?



DW vs. Big Data

Are **Data Warehouses** a solution for Big Data apps?

Points in common

- Dealing with **massive** datasets
- Integrating **heterogeneous sources**

Yet ... :

- Data Warehouses are typically **centralized**
- Data Warehouses typically use **structured data**

Distributed Computing



In many applications that must deal with Big Data, it is possible to exploit **parallelism** in computation.

Some examples may be:

- ranking documents (compute scores in parallel)
- search friends in social networks (explore paths)

In such cases, computing paradigms also change, moving from supercomputers to **computing clusters**

Distributed Computing



The **distributed computing** (or ***cluster computing***) is the method of making multiple computers work together to solve a common problem [[AWS](#)].

The distributed computing paradigm presents some key characteristics:

- data is stored **redundantly** across nodes;
- computation is divided into **tasks**;
- a **distributed file system** is exploited.

Advantages

 Improved Security

 Flexibility and Scalability

 Enhanced Performance

 Low Latency

 Increased Reliability

 Expanded Storage

 Cost effectiveness

Distributed Computing

 Maintenance Fees

 Component Failure

 Bandwidth Restrictions

 Systems Complexity

 Development Obstacles

 Slow Network Transfers

 Issues with Standardization

Disadvantages

Distributed File System



A *distributed file system* (**DFS**) is a file system that is specifically designed for cluster computing:

- File size can be **huge** (e.g., Terabytes)
- Files are **rarely updated** (persistent data are typically loaded and used for computations)
- Files are usually split into **chunks**
- Chunks are **replicated** at multiple (e.g., 3) nodes
- A **master node** knows where chunks are located
- Different models are exploited to handle and manage the system (***master-slave***, ***p2p***)

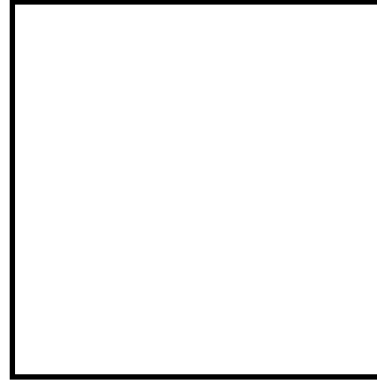
Key characteristics can be summarized into 3 concepts:

- **Master-Slave**: replicate data across multiple nodes
- **Peer-to-Peer**: no master, all nodes are treated as equal
- **Sharding**: different data portions onto different servers

Distributed File System



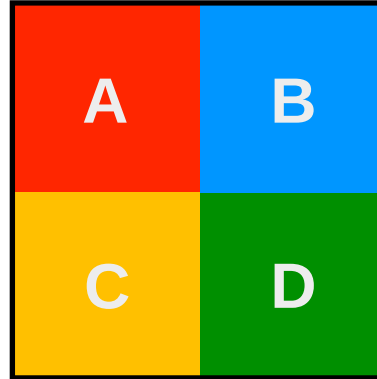
Large File



Slide Idea
from J. Gonzalez
(Berkeley)

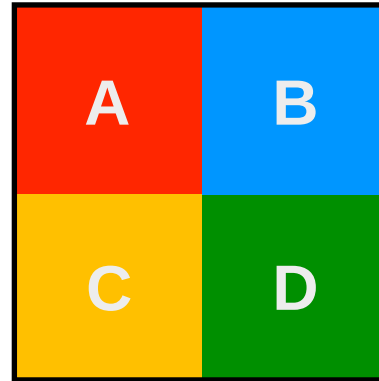
Distributed File System

Large File
split into
CHUNKS

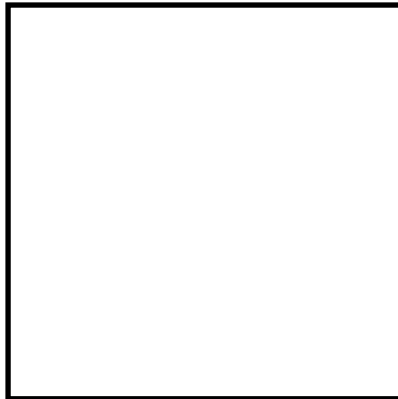


Slide Idea
from J. Gonzalez
(Berkeley)

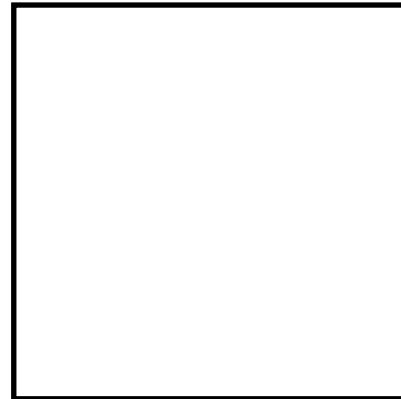
Sharding



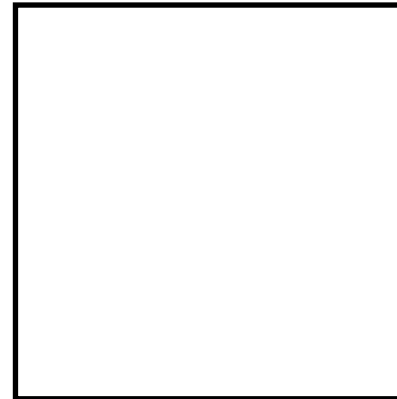
Slide Idea
from J. Gonzalez
(Berkeley)



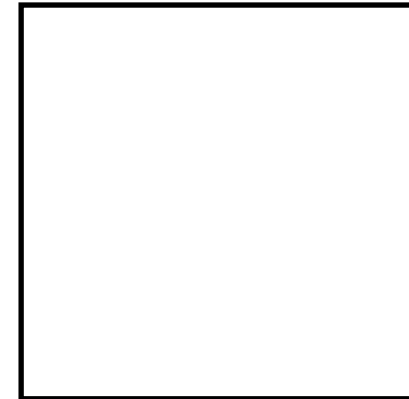
Node 1



Node 2



Node 3

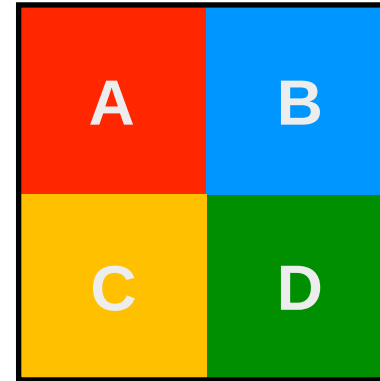


Node 4

Sharding

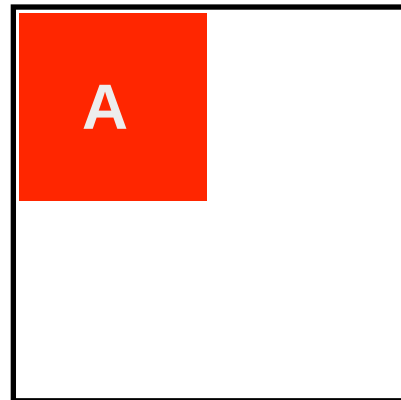
Additional aspects to be considered:

- Decide how to **clump** data (so that user typically access a single node rather than multiple ones);
- **Balance load** across the nodes;
- Places data **close** to where it is accessed.

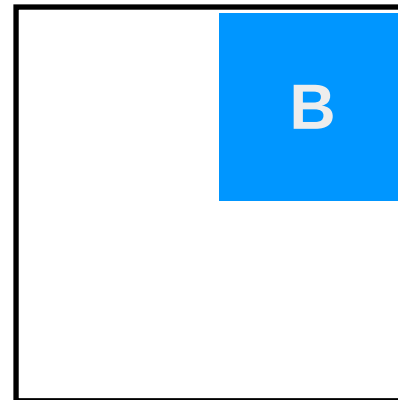


Slide Idea
from J. Gonzalez
(Berkeley)

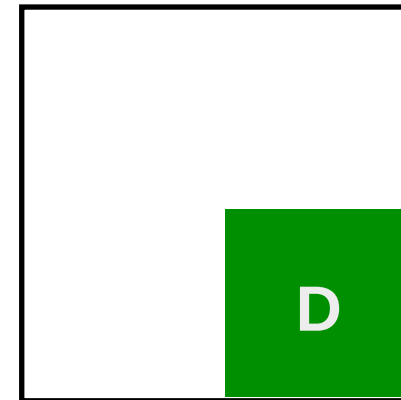
The approach can be used in **combination** to the master-slave architecture.



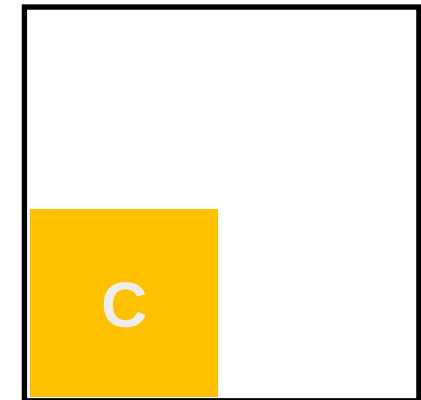
Node 1



Node 2



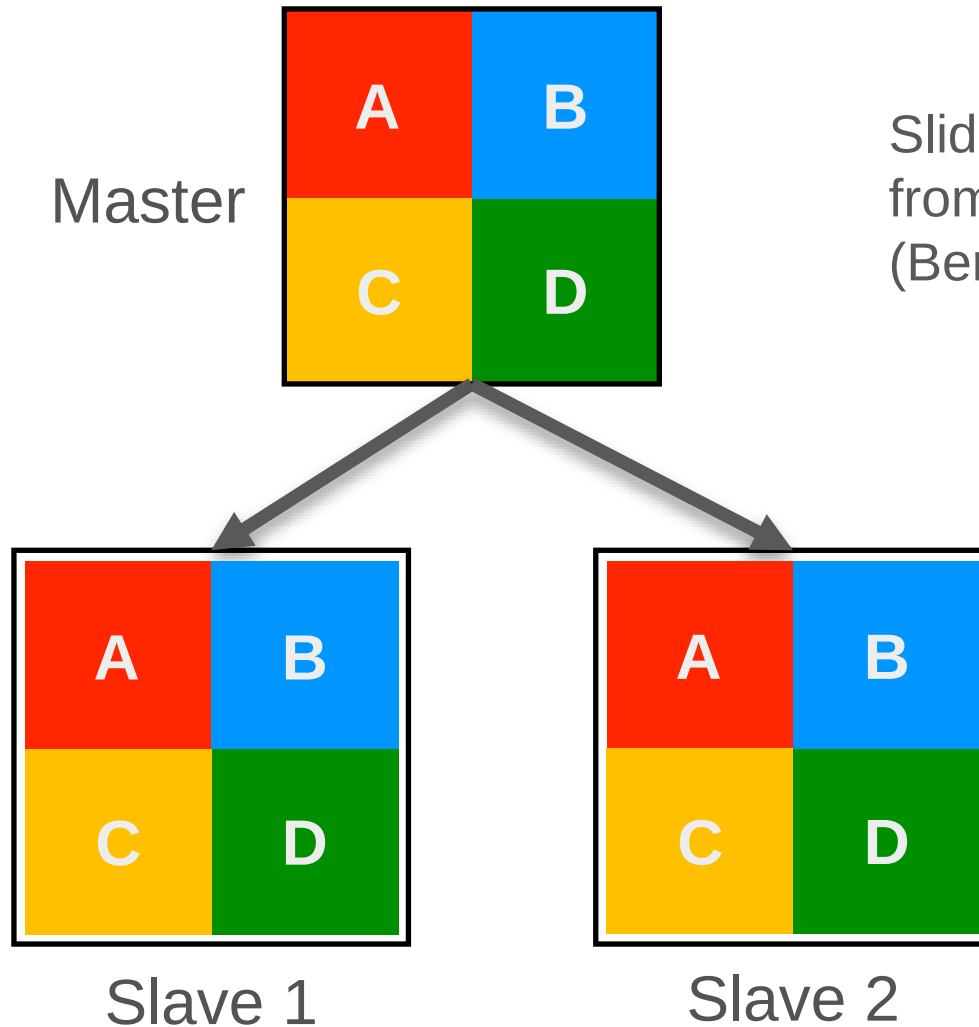
Node 3



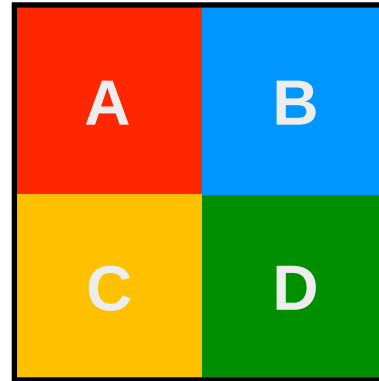
Node 4

Master-Slave

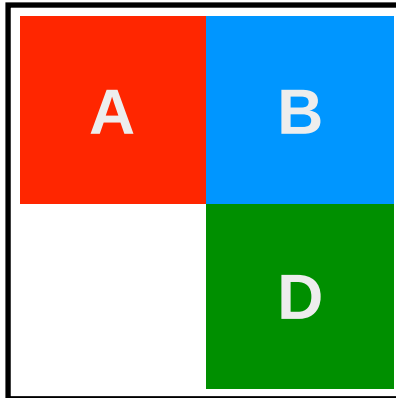
- The node designated as **master** (or ***primary***) is the **authoritative** data source, and it is **responsible** for processing any updates to that data
- Other nodes are **slaves** (or ***secondaries***)
- A **synchronization process** is needed to update slaves according to the master status
- Single point of write/update
- Single point of **failure**



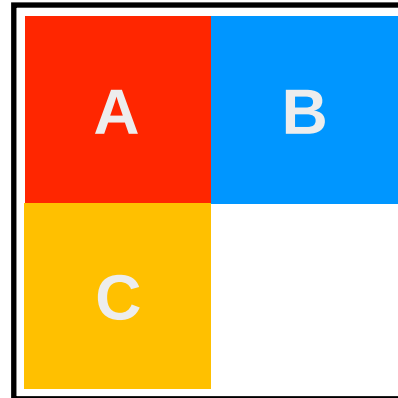
Master-Slave & Sharding



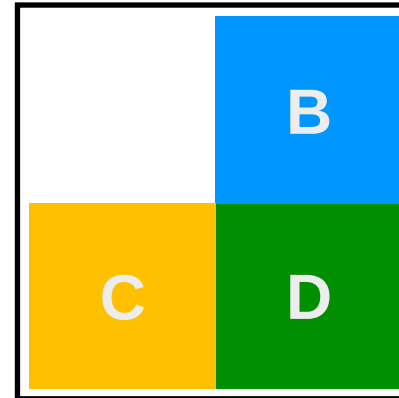
Slide Idea
from J. Gonzalez
(Berkeley)



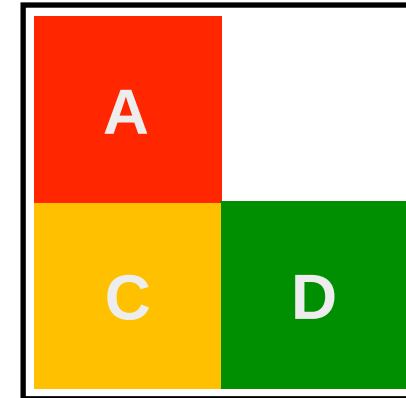
Node 1



Node 2



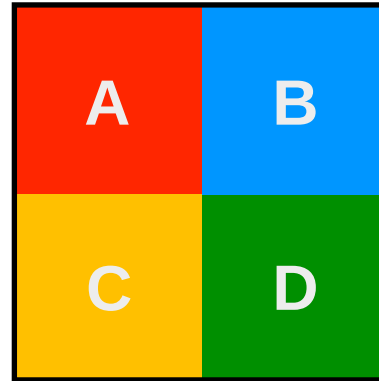
Node 3



Node 4

Master-Slave & Sharding

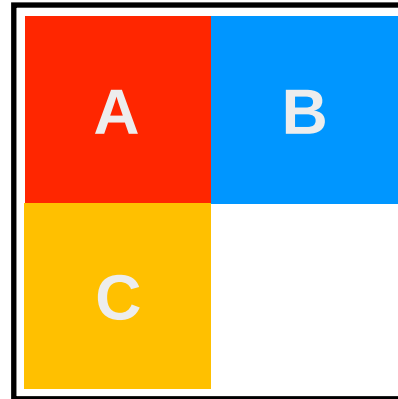
Fault Tolerant!



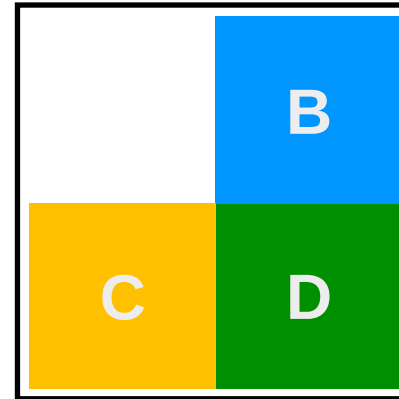
Slide Idea
from J. Gonzalez
(Berkeley)



Node 1



Node 2



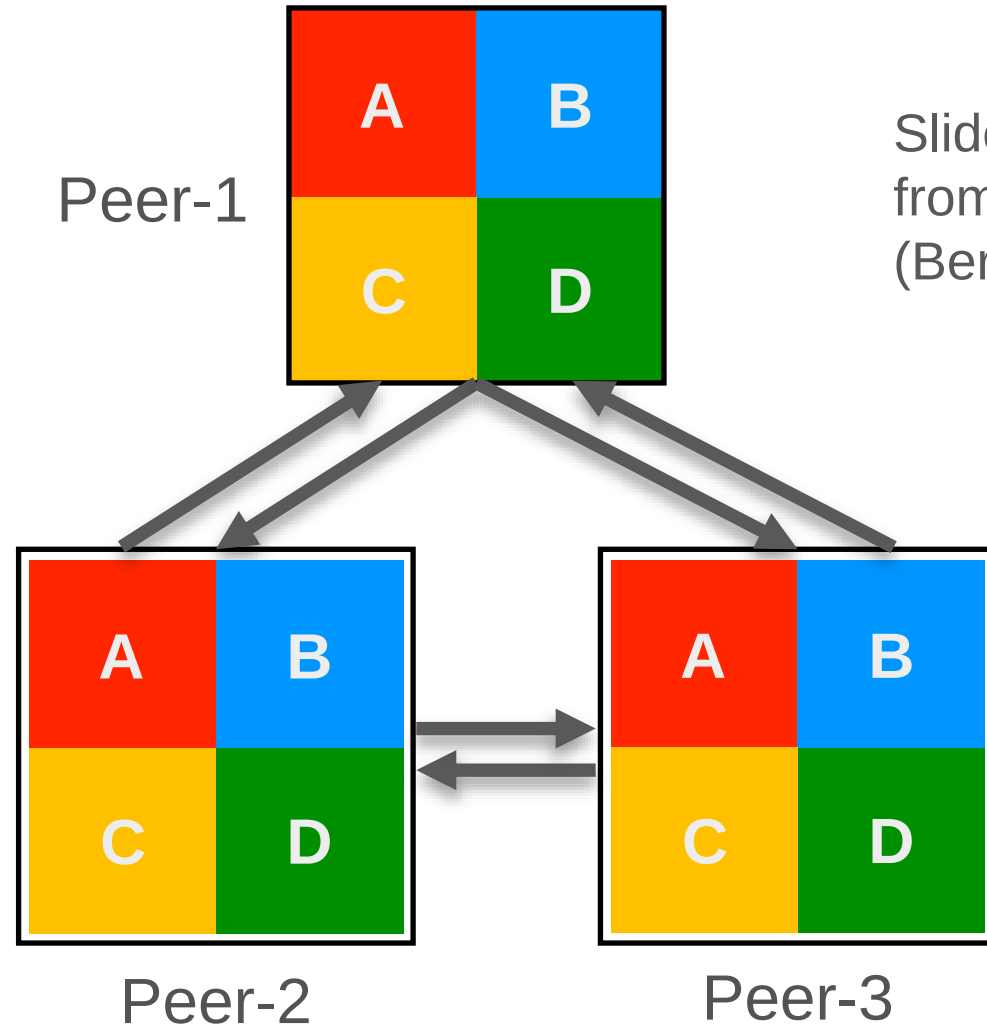
Node 3



Node 4

Peer-to-Peer

- Architecture similar to master-slave, but **without** an authority node with more duties and responsibilities
- **No more** a single point of failure
- Fault tolerance is **improved**
- Replicas have to **coordinate** one another



Slide Idea
from J. Gonzalez
(Berkeley)

ACID Properties

Traditional relational databases must enforce **strong consistency** properties (so-called ACID)

- **A**tomicity — transactions executed as all-or-nothing
- **C**oherence(*) — transactions preserve constraints
- **I**solation — transactions do not interfere
- **D**urability — after commit, data become permanent

(*) Consistency often used instead of Coherence

ACID Properties



In traditional RDBMS:

- Atomicity **always** guaranteed
- Coherence **enforced** when the transaction commits
- Durability is **always** guaranteed
- Isolation is the **most crucial** property to guarantee

ACID Properties



Problems with guaranteeing isolation:

- **Read** operation of data that **is being written**
- **Write** operation of data that **is being written**
- **Write** operation of data that **is being read**

These are typically solved with LOCK operations...

CAP Properties

CAP Theorem (also known as Brewer's Theorem)

A distributed data system **cannot** simultaneously provide more than two of the following properties:

- **C**onsistency:
 - Read operations get the **most recent** data;
- **A**vailability:
 - Every node (not failed) **always** executes queries;
- **P**artition Tolerance:
 - The system **continues operating** correctly also in the presence of node **failures**.

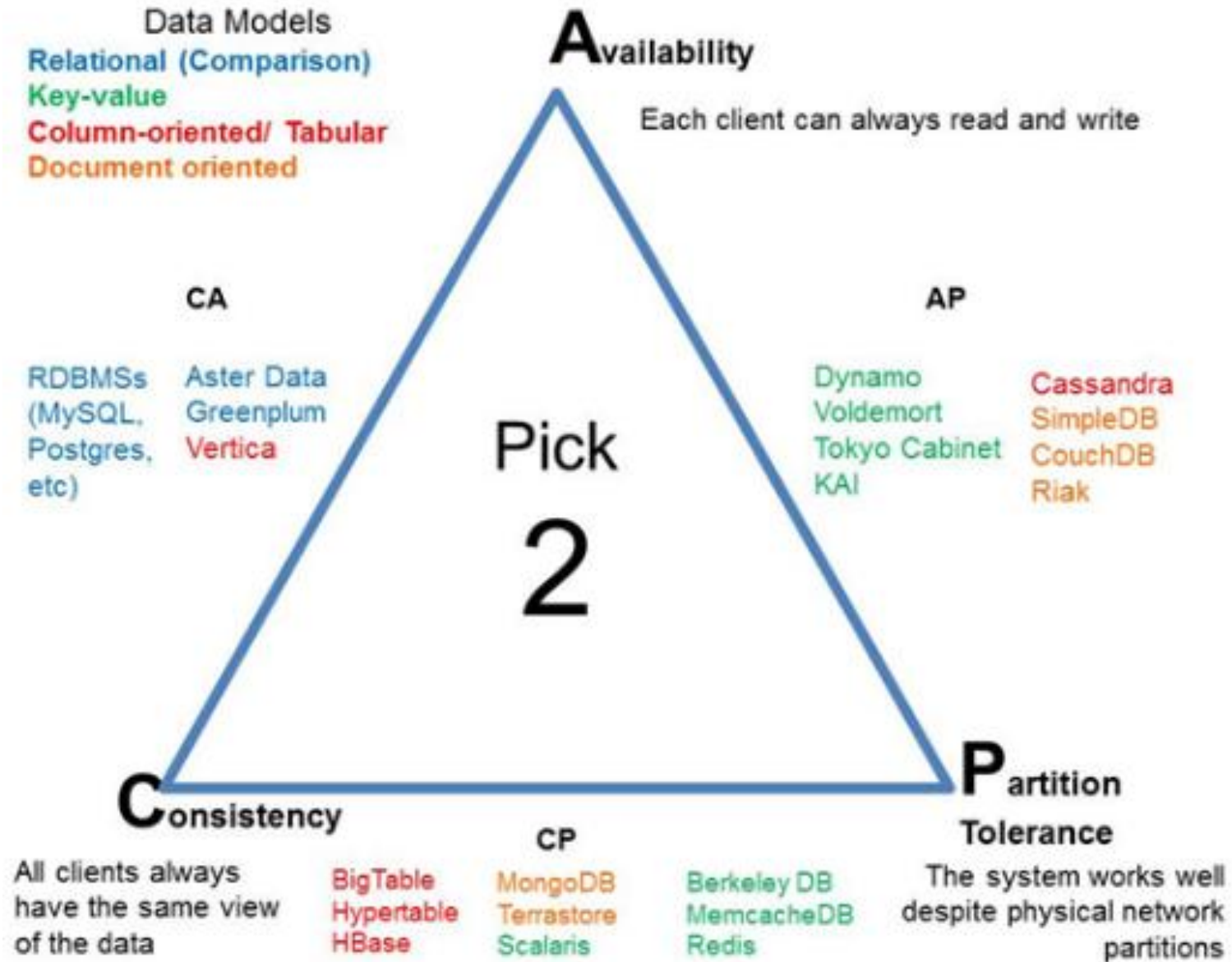
CAP Properties

CAP Theorem: Consequences

Since Partition Tolerance has **almost always** to be guaranteed, one must **choose** between either Consistency or Availability

Basically, this means that **not all the nodes** can be **simultaneously** updated with the most recent data

CAP Properties



Distributed File System



Examples of distributed file systems include:

- Google File System (GFS)
- Hadoop DFS (HDFS)
- Cloudstore
- ...

<https://github.com/theanalyst/awesome-distributed-systems>

NoSQL (*Not Only SQL*)

Historically, **relational databases** have been the main paradigm for data storage in business applications.

The advent of big data has switched the attention towards data that **evolve more rapidly**, where traditional relational databases show some limitations.

The NoSQL (Not Only SQL) movement encourages the use of **alternative solutions** for data storage

Limits of RDBMS

Limits of RDBMS in the context of Big Data:

- Need to design the **static** structure in advance
- Difficult to handle **high variety** or **non-tabular** data
- **Not** conceived for **distributed** solutions
- Very **long** execution times
- The so-called “**join pain**”

Most widely used NoSQL paradigms:

- Graph databases
- Key-value databases
- Document databases
- Column family databases

Graph Databases

A **graph database** is a database that uses graph structures to represent the data

- It is **schema-less**
- It does **not** have **tables**
- **Nodes** represent **entities**
- **Edges** represent **relations**

They can be used for both OLAP and OLTP

Graph Databases

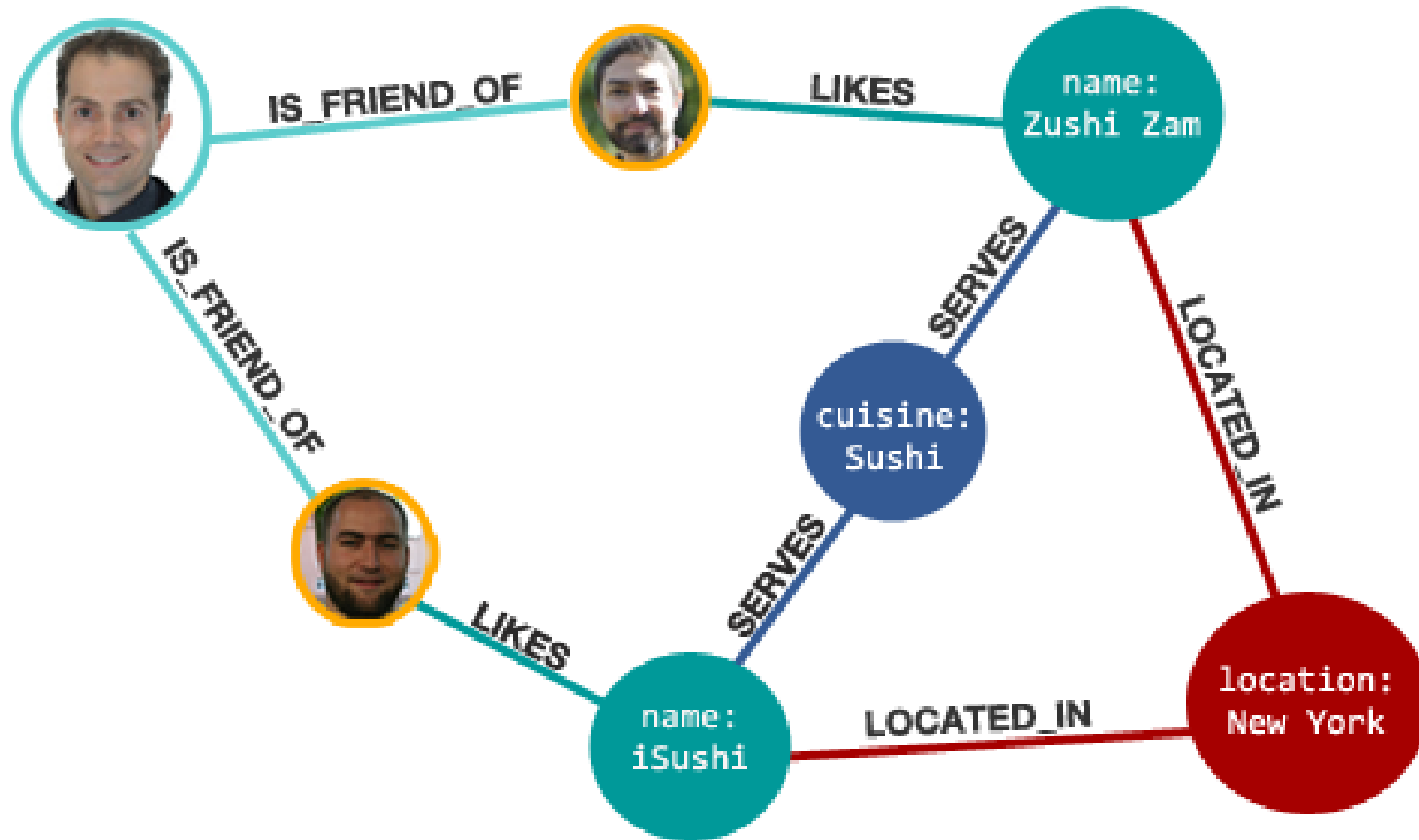


Figure from neo4j.com

Graph Databases



Advantages:

- Accumulate data **incrementally**
- **Flexibility** in data representations
- Efficiently manage **sparse** data
- Naturally represent **domains** with graph structure
- Naturally represent **relationships**

Graph Databases



Performance:

- Queries through **specific** languages (*Cypher*, from Neo4j, is the **most popular**, like SQL)
- **Avoid classic join** (costly in performance, especially with a big amount of data)
- Depending on the underlying **storage engine**
- Depending on the **kind of queries**:
 - More efficient on navigation, relationship analysis, path analysis and distance analysis queries; if instead, queries are more oriented towards analysis of a single node or attributes, performance are not significantly better with respect to RDBMS

Graph Databases

Example — Friends-of-friends with traditional RDBMs

PERSONS

id	name
1	Alice
2	Bob
3	Carl
4	David
5	Emily

FRIENDS

id	id_friend
1	2
2	1
3	4
4	3
5	1
1	5

```
SELECT * FROM Persons P1
JOIN Friends F ON P1.id=F.id
JOIN Persons P2 ON P2.id=F.id_friend
```

**RETRIEVE
FRIENDS**

Graph Databases

Example — Friends-of-friends with traditional RDBMs

PERSONS

id	name
1	Alice
2	Bob
3	Carl
4	David
5	Emily

FRIENDS

id	id_friend
1	2
2	1
3	4
4	3
5	1
1	5

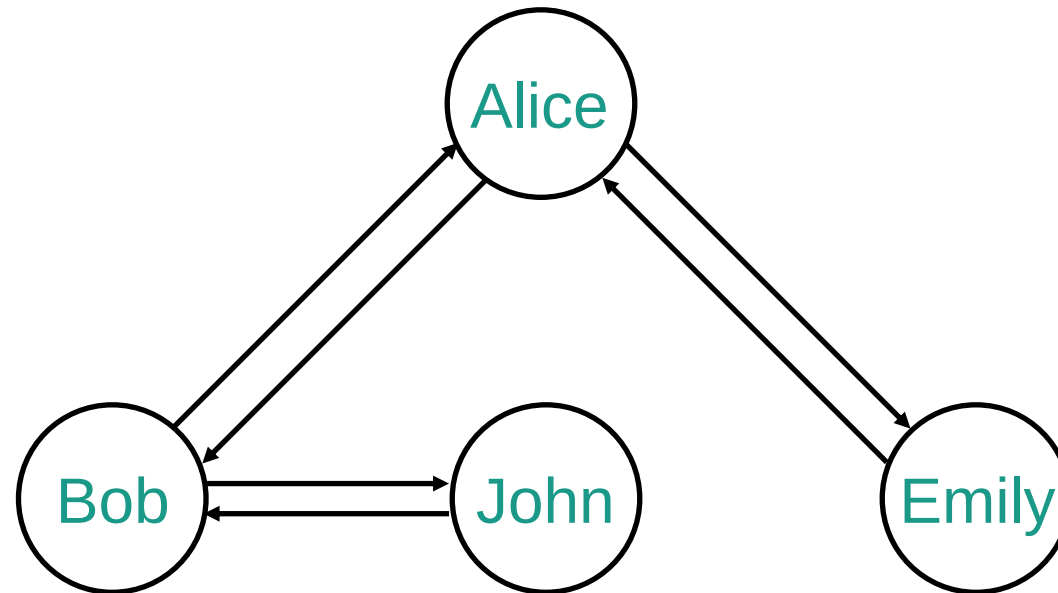
```
SELECT * FROM Persons P1
JOIN Friends F1 ON P1.id=F1.id
JOIN Friends F2 ON F2.id=F1.id_friend
JOIN Persons P2 ON F2.id=P2.id
WHERE P1.id<>P2.id
```

**RETRIEVE
FRIENDS
OF FRIENDS**

Graph Databases

Example — Friends-of-friends with traditional graph DBs

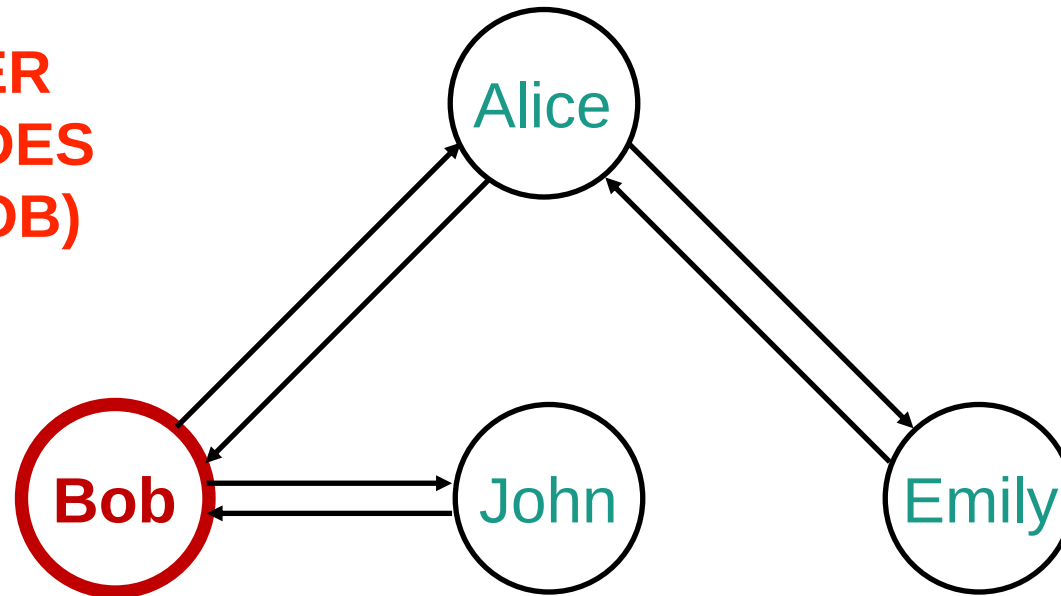
QUERIES ARE
COMPUTED AS
PATH-BASED
OPERATIONS



Graph Databases

Example — Friends-of-friends with traditional graph DBs

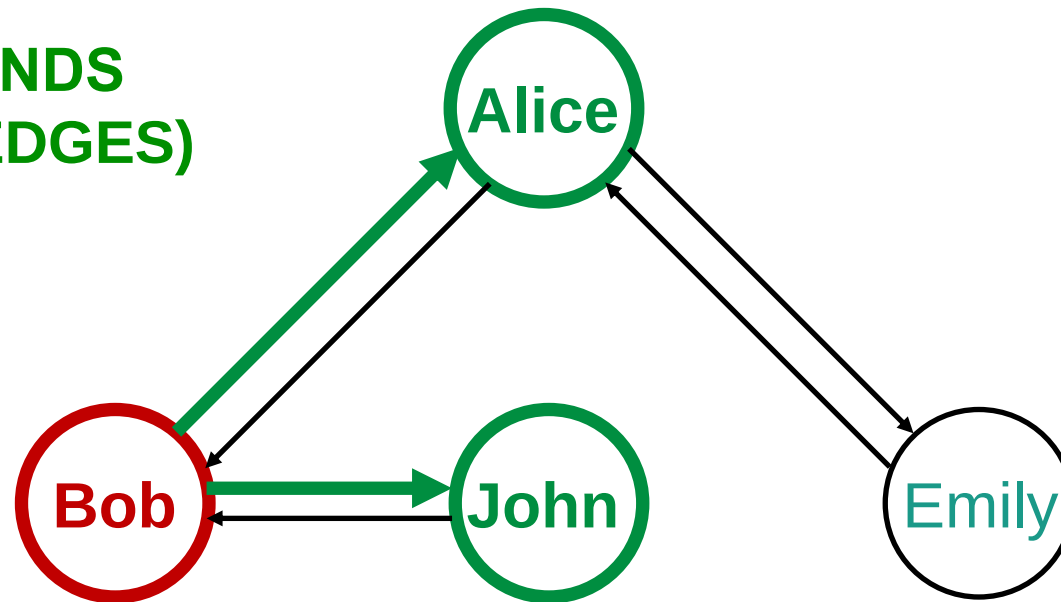
**ITERATE OVER
ALL THE NODES
(e.g. TAKE BOB)**



Graph Databases

Example — Friends-of-friends with traditional graph DBs

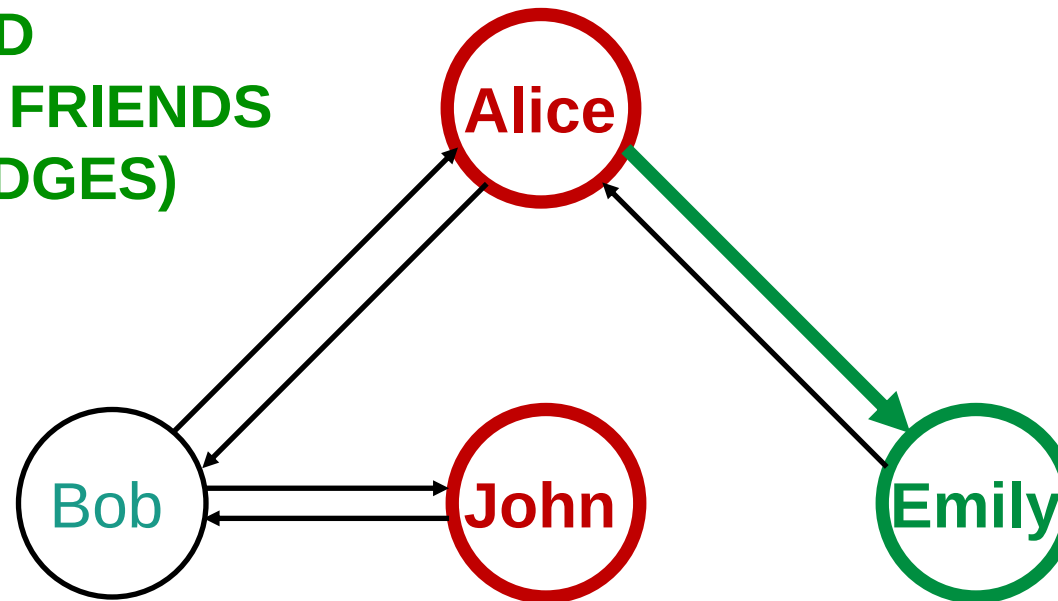
**FIND BOB'S FRIENDS
(JUST FOLLOW EDGES)**



Graph Databases

Example — Friends-of-friends with traditional graph DBs

**FOR EACH FRIEND
(e.g., ALICE) FIND FRIENDS
(JUST FOLLOW EDGES)**



Graph Databases



Time comparison

Depth	RDBMS execution time(s)	Neo4j execution time(s)	Records returned
2	0.016	0.01	~2500
3	30.267	0.168	~110,000
4	1543.505	1.359	~600,000
5	Unfinished	2.132	~800,000

Table from “Neo4j in Action”

Graph Databases



Breadth first search

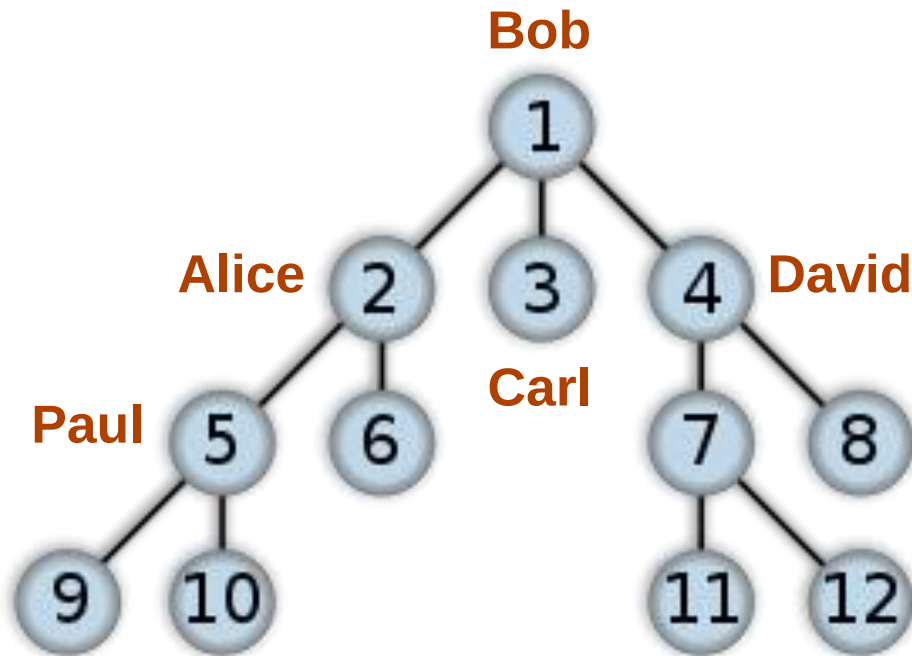
- Expand **shallowest** unexpanded nodes first
- Store unexpanded nodes in a **queue**

Depth first search

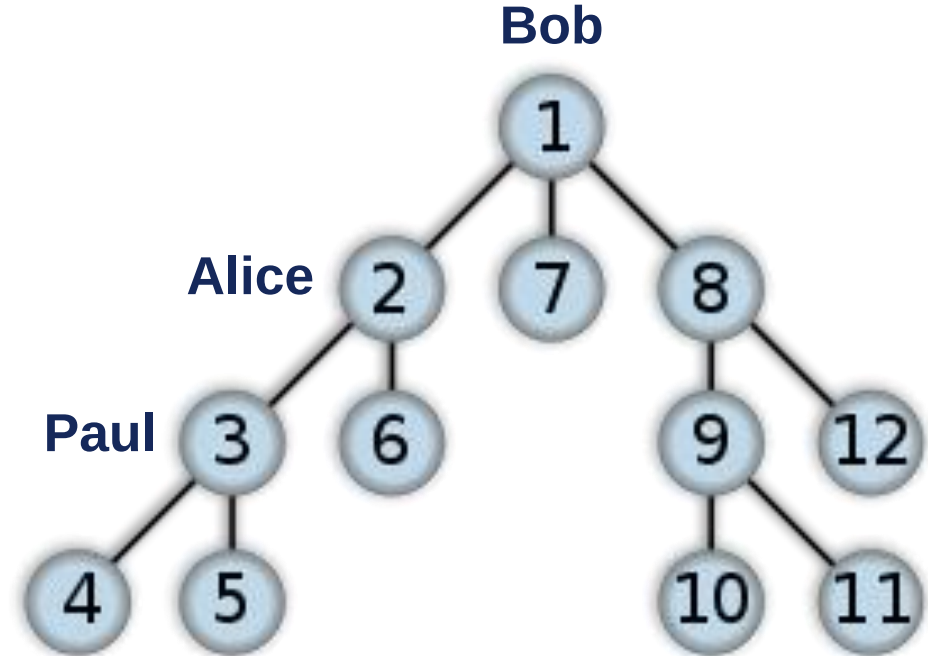
- Expand **deepest** unexpanded nodes first
- Store unexpanded nodes in a **stack**

Graph Databases

Queries are executed as **traversal searches**



Breadth-First Search



Depth-First Search

[Images from Wikipedia]

Graph Databases



Software:

- Neo4j
- OrientDB
- ArangoDB
- Titan
- ...

Aggregate Databases

Aggregate databases are based on the idea that some data can be **aggregated** and **indexed** by a key.

Why?



Because **data units** are not necessarily tuples (rows) as in relational databases, but they may be data with a **more complex structure**.

- Indeed, oftentimes one must operate with data with a **complex** structure (e.g., lists, nested records).
- Additionally, to deal with big data, models should **scale up** to many machines (**distributed** systems).
- Key-value, document, and column-family databases all fall into the category of **aggregate databases**.

Key-Value Databases

A key-value database is a sort of big **hash map** that associates to each key a value:

- Data can be accessed **only via that key**;
- The **value** acts as an **aggregate** of information;
- Basically, **anything** can be stored in the value;
- The value is an **opaque** (it cannot be partitioned).

Data access and retrieval:

- Access data **efficiently** always via primary key;
- Values are just sequences (**blobs**) of bits;
- It is the responsibility of the **application** to understand what/how is stored in a value.

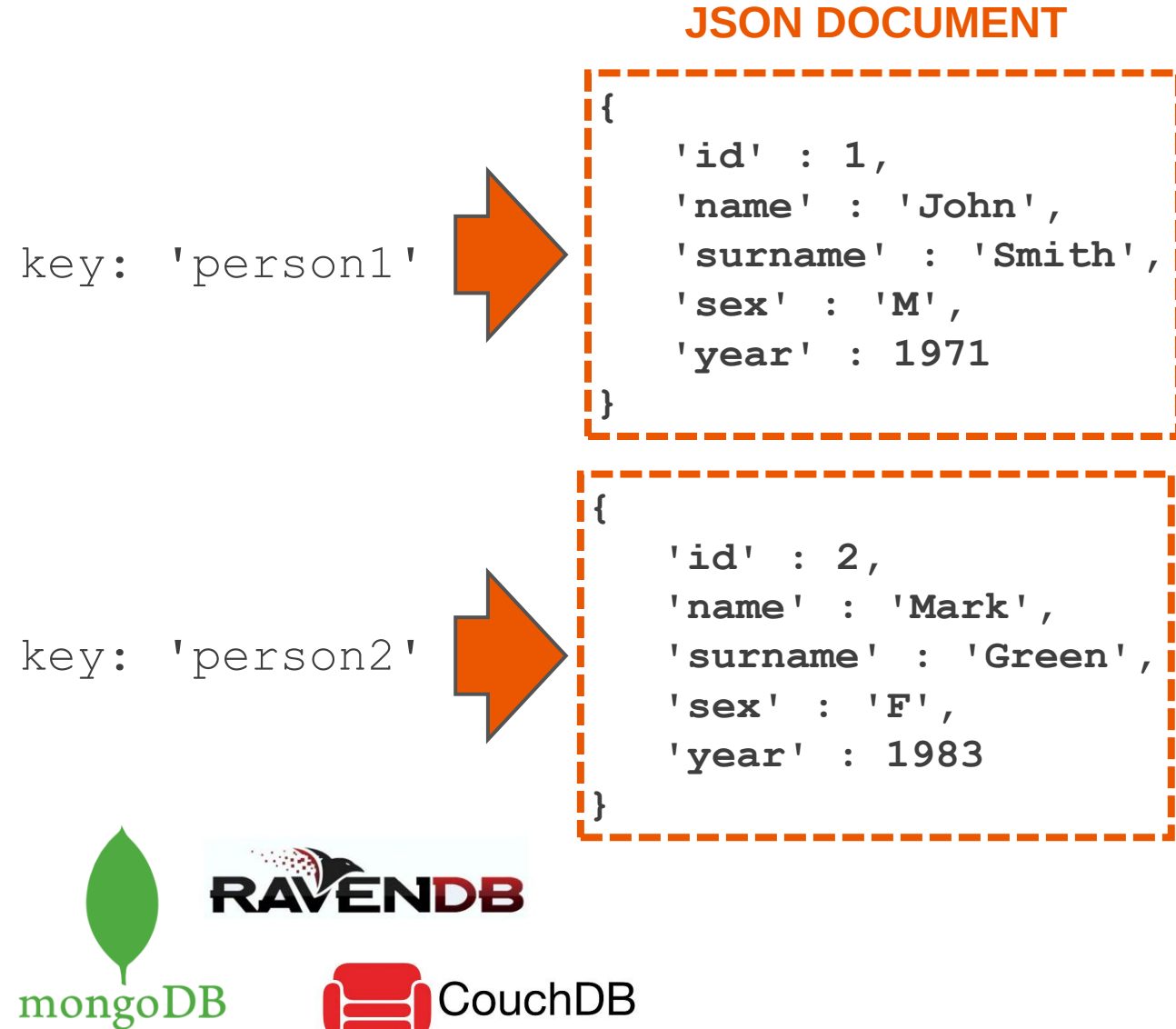
Key	Value
1	John; Smith; M; 1971
2	Mark; Green; M; 1983
3	Alice; Collins; F; 1987
4	Jack; Russel; M; 1955
5	Mary; Stewart; F; 1960



Document Databases

Like key-value databases, but the association is between a key and a document (e.g., a JSON file):

- Differently from key-value, values have a **structure**;
- Yet, there are **limits** in what is stored in the value;
- In principle, one could **query also value attributes** as well as retrieve only **part** of the value.



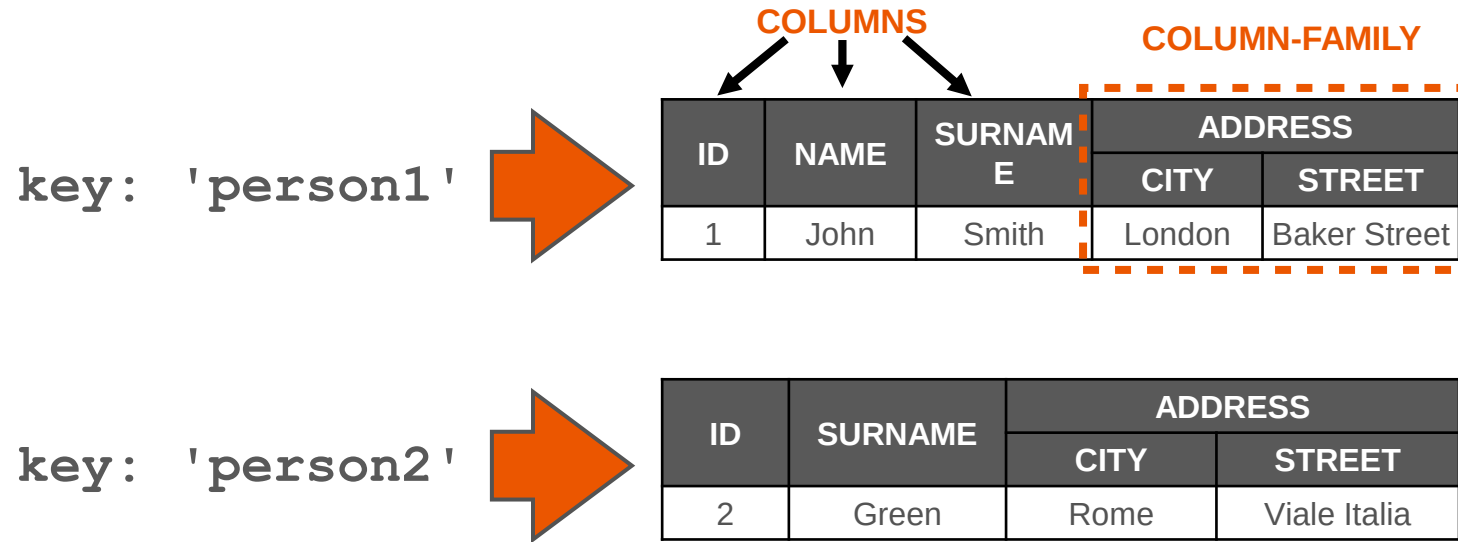
Column Family Databases

Another variant quite like key-value databases:

- For each key, values are stored in **columns**, that can possibly be grouped into **column families**;
- Tables are **sparsely** populated;
- Query can be made **by key only**.



Column Family Databases



Data structure can be seen as row or column oriented:

- A **column family** contains a collection of **rows**;
- A **row** contains a collection of **columns**;
- **Not necessarily** all the records store the **same** columns: they can use different names/types;
- For each value, a **timestamp** is sometimes stored.

Note: this is the terminology of *BigTable* by Google, whereas Cassandra uses a slightly different notation, speaking of super-columns

Software



Key-value databases:

- Riak, Redis, Amazon DynamoDB, ...

Document databases:

- MongoDB, CouchDB, RavenDB, ...



Column-family database

- BigTable, Cassandra, Hbase, Amazon SimpleDB



Aggregated Databases: to summarize ...

AGGREGATE DATABASE	MAIN FEATURES
Key-Value Databases	The aggregate is treated as a whole. Efficient queries on primary key, effort on the applications.
Document Databases	The aggregate is structured, allowing queries and partial retrieval. Documents do not have schemas, thus retrieval not easily optimized.
Column-Family Databases	The aggregate is structured through column families, treated as data units. Accessibility can take advantage of this structure (less flexibility).

Not having to **define schemas in advance** is a nice feature that allows flexibility in data storage. Yet, when writing an application, you often **must know** which are the data stored in your database. We speak in this case of **implicit schemas**.

SQL vs. NoSQL

Dealing with unstructured information

- SQL needs a (complex) table **structure**
- NoSQL can exploit **unstructured** data easily
- Web data is **inherently** unstructured

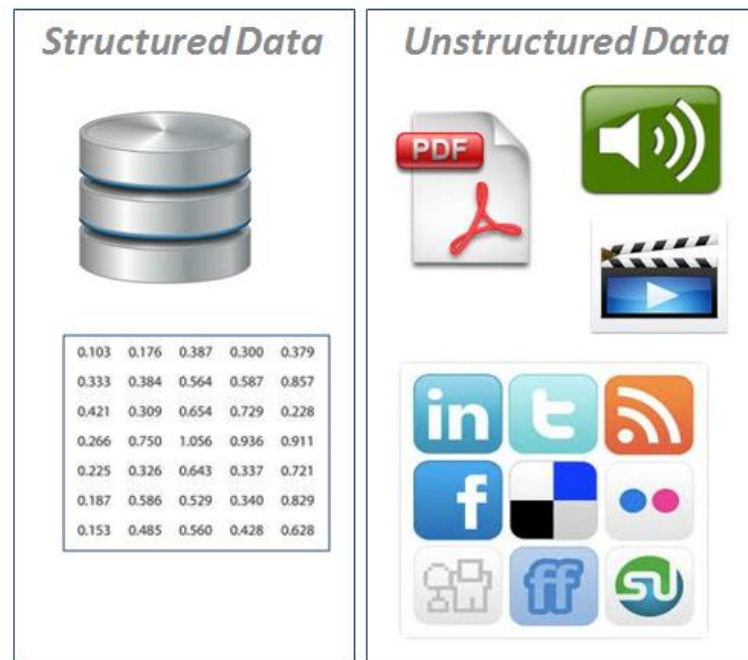


Image from
www.laserfiche.com

SQL vs. NoSQL

Storage costs:

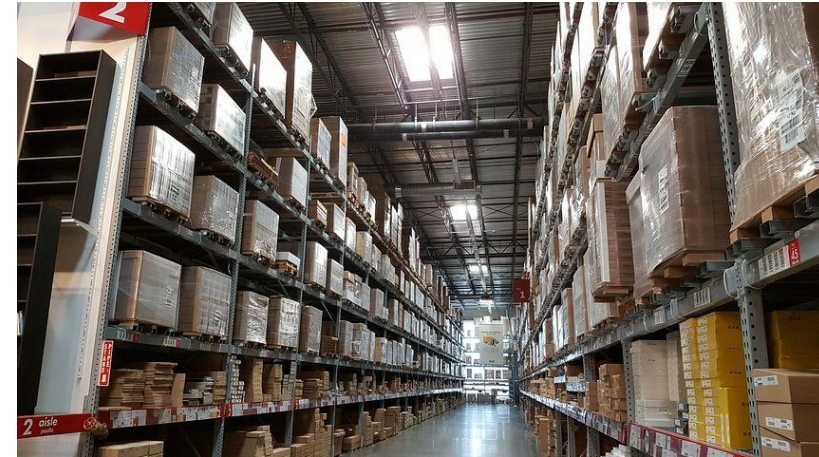
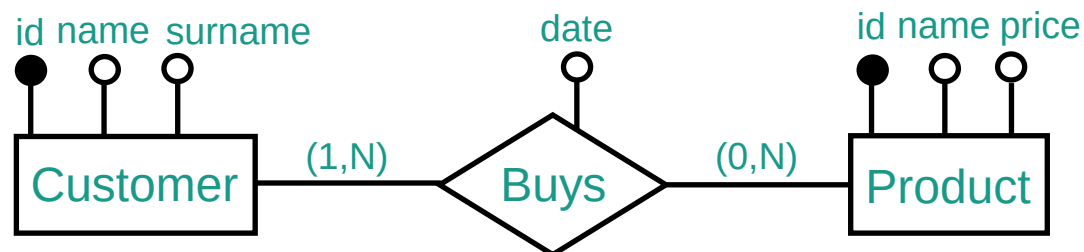
- SQL tables need **normalization** to avoid duplicates;
- NoSQL **exploits duplication** to improve reliability.

Computational costs:

- SQL databases scale with **extra system resources**;
- NoSQL databases scale with **more clusters**.

Design and modeling:

- SQL databases have **well-established procedures**;
- NoSQL databases do not (are young approaches).



SQL vs. NoSQL

Integrity and consistency properties:

- **ACID vs. CAP**

NoSQL databases typically do **not** support ACID properties, but this is not completely true... :

- Graph databases **do support** ACID transactions!

For the other cases, need to **relax** consistency:

- **Prevent** conflicts vs. **correct** conflicts
 - **Prevent conflicts:** Do not allow operations that produce a conflict;
 - **Correct conflicts:** Tolerate operations that produce a conflict, but detect them, and fix the conflict afterwards.

MapReduce

MapReduce is a **style of computing** that has been implemented in many systems (Google, Hadoop, Spark) to exploit *distributed computation* for big data.

Originally developed by **Google** in 2003-2004.

It is based on the definition of **two functions**:

- **Map**
- **Reduce**



MapReduce



Any MapReduce algorithm receives in input a set of instances of a single type (e.g., documents).

A **Map** function (written by the user) is applied to each input instance **in parallel** by a set of computational units (called *mappers*) and it produces as output a set of **key-value pairs**.

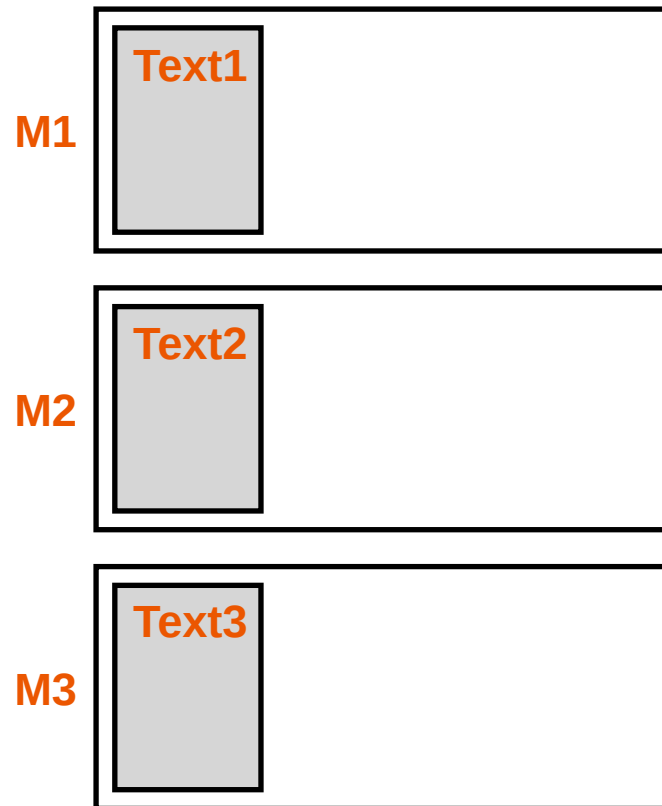
All the outputs are **sorted by key**, to obtain a set of **key-(list-of-values)**.

A **Reduce** function is then applied to **each** key and its corresponding list of values, to compute an output. This is also **parallelized** over a set of computational units named *reducers*.

The final output of the algorithm is the **concatenation** of all the outputs of the Reduce functions.

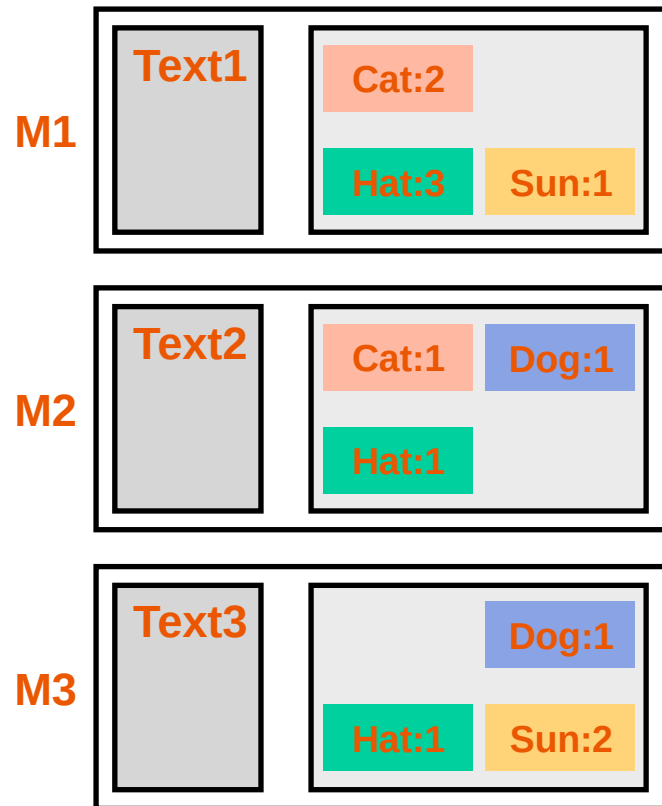
MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



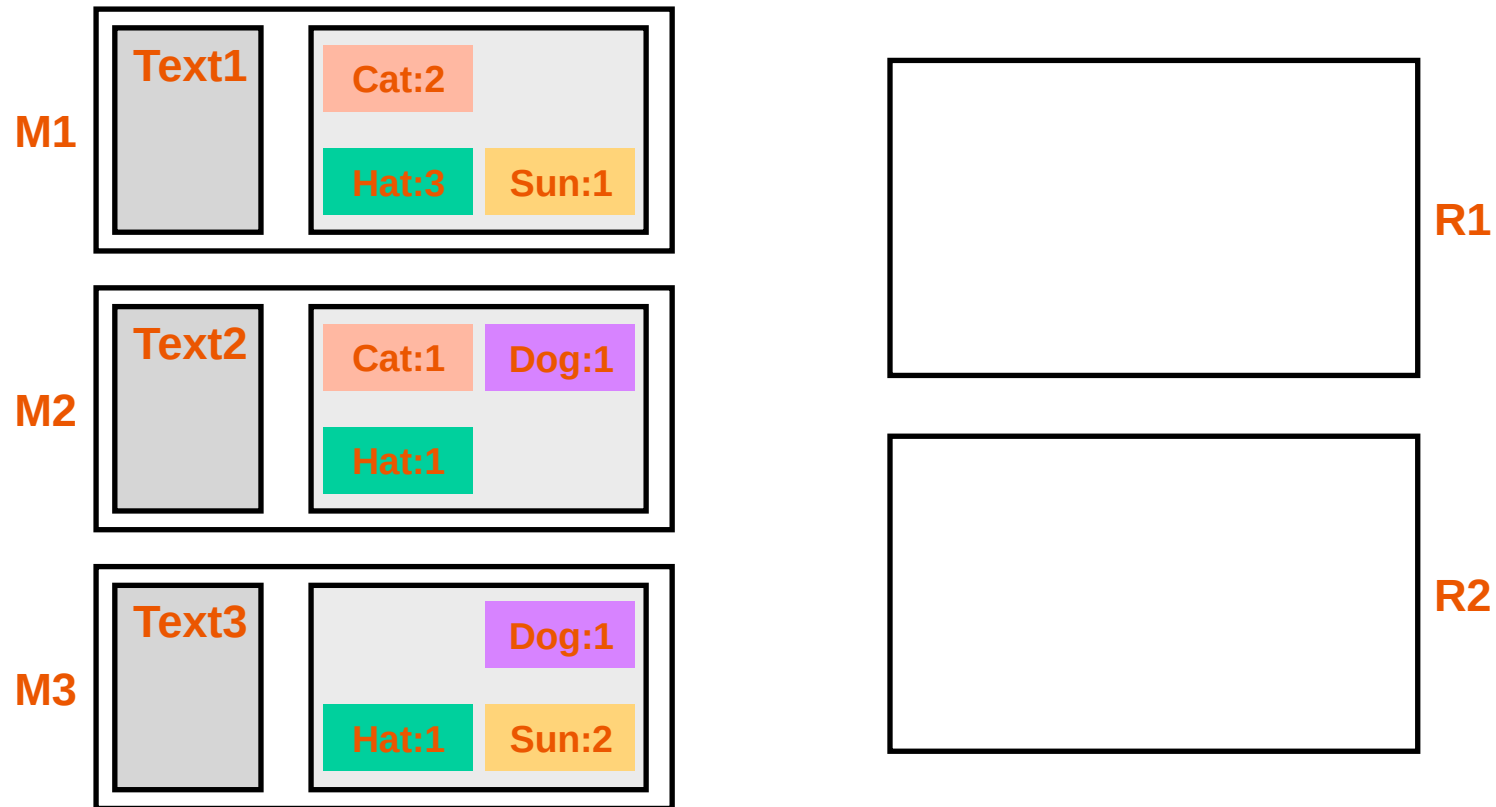
MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



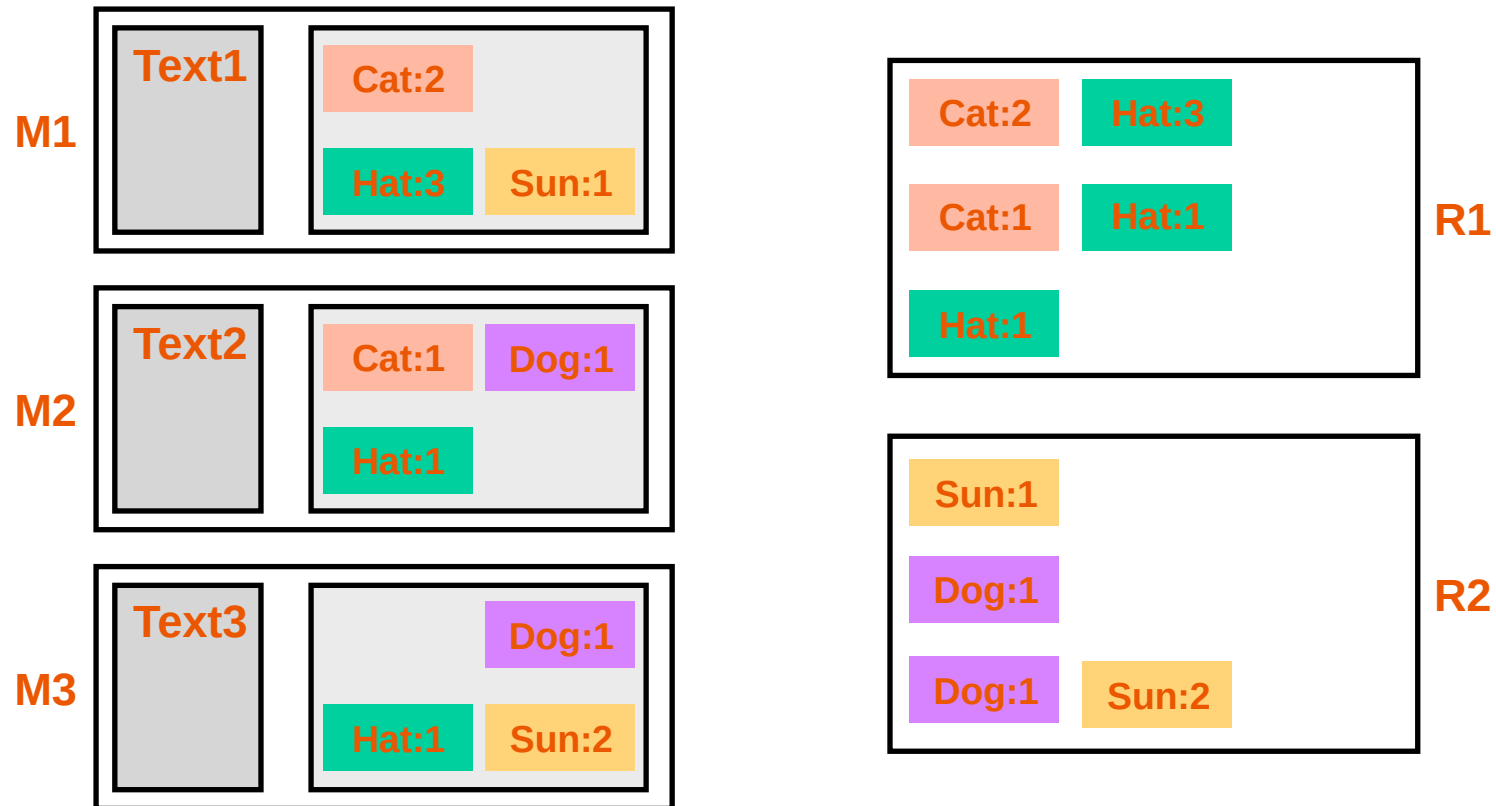
MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



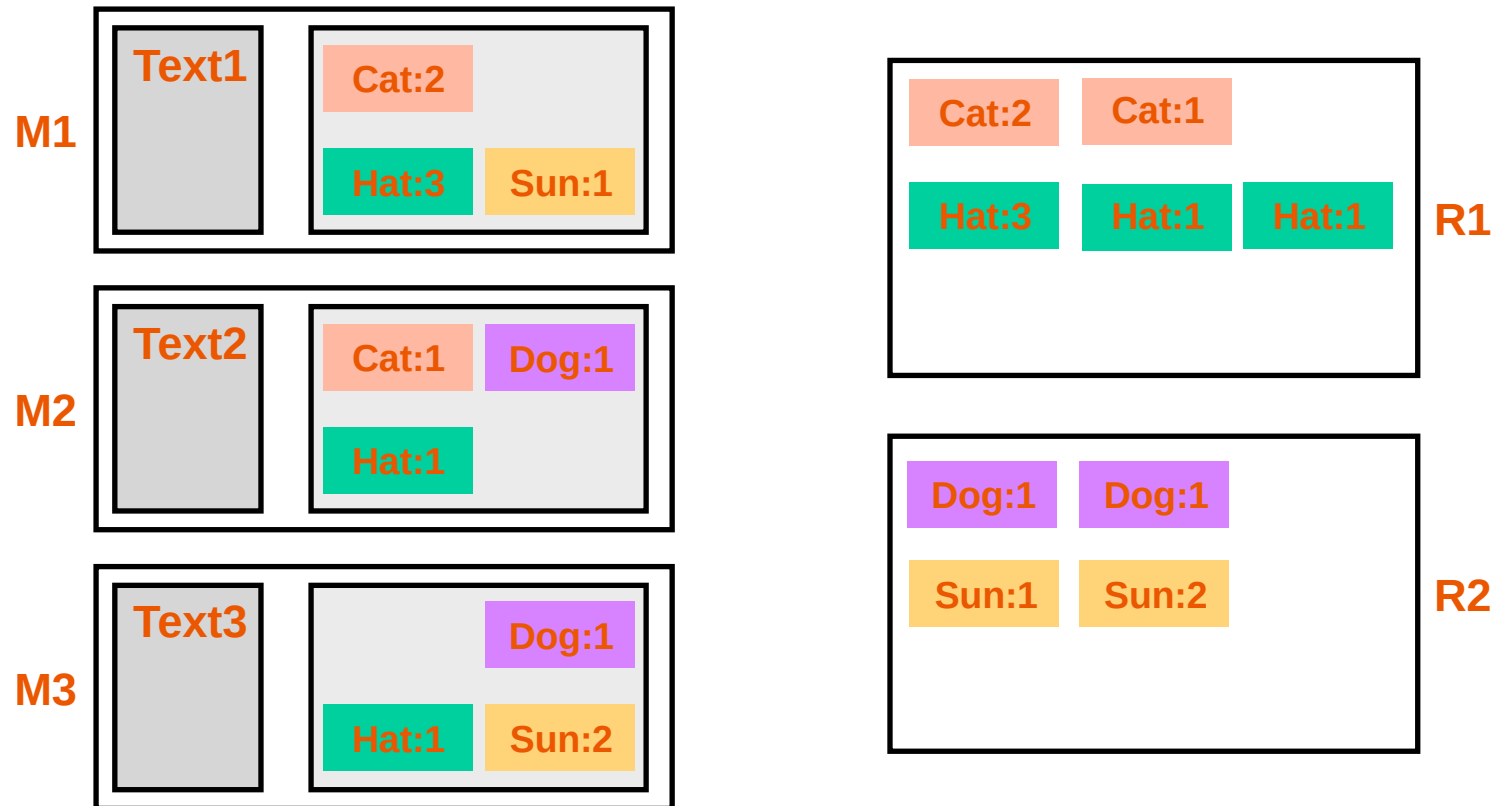
MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



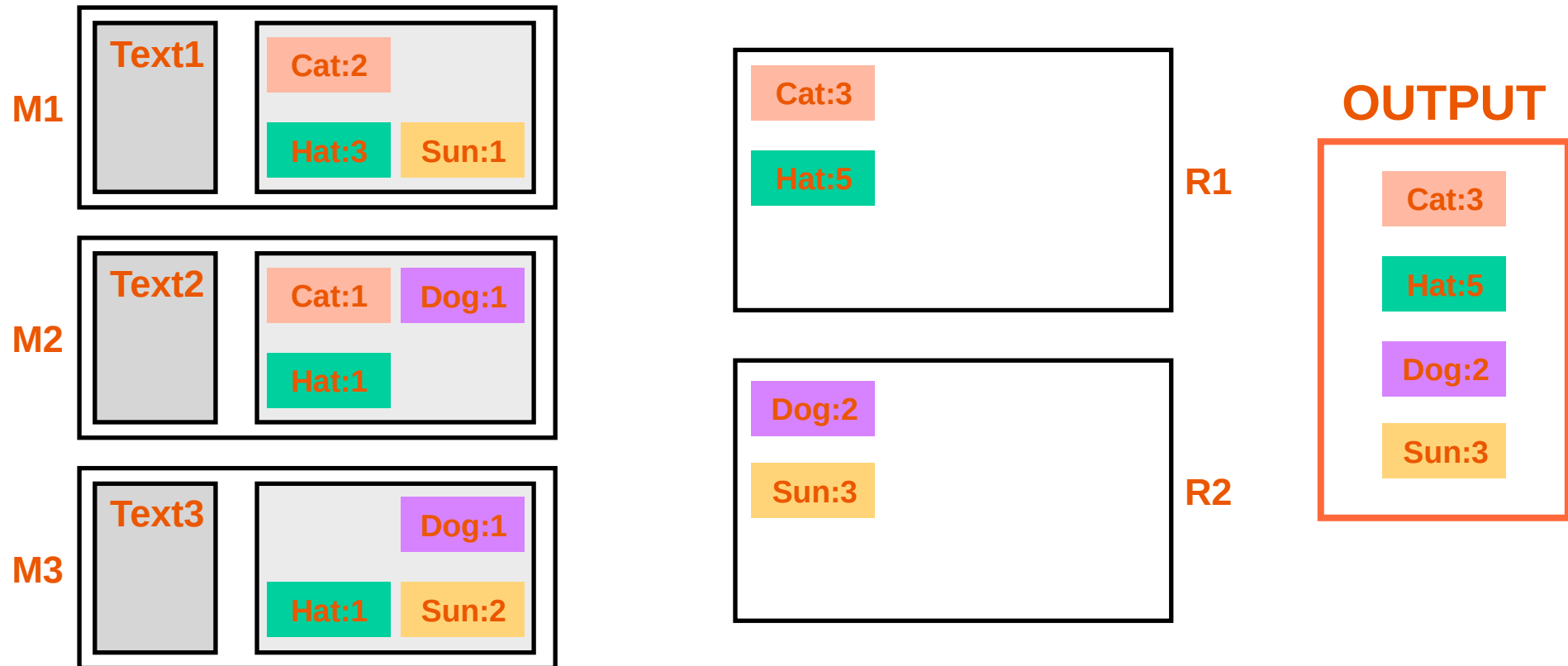
MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



MapReduce

Example - Word count: Given a large set of documents, **count the occurrences of each distinct word** in the corpus.



MapReduce

Further extensions

- Several MapReduce operations in **cascade**
- The **output** of a Reduce as **input** to another Map

Hadoop Platform

- Developed by Apache
- Extremely popular
- **Open-source** implementation of MapReduce
- Based on **Java** and **Scala** programming languages

